

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
21 };
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

2472 · Maximum Number of Non-overlapping Palindrome Substrings

LeetCode · Hard

Hard

DP

Palindrome

Greedy Lemma

1. What Type of Question Is This?

This is a **1-D interval-selection DP** problem — the same family as "select the maximum number of non-overlapping items from a sequence", except the "items" here are palindromic substrings you have to discover as you go, rather than being handed a ready-made list. The DP shape ($dp[i]$ built from earlier dp values) is standard, but the problem only becomes tractable once you find a genuinely clever shortcut: at any ending position, you never need to check every possible palindrome length — only the two shortest legal ones.

2. What Is Given?

- A string **s** and a positive integer **k**.
- You must select a set of **non-overlapping** substrings of **s** such that every selected substring is a **palindrome** and has **length $\geq k$** .
- A palindrome reads the same forwards and backwards; a substring is any contiguous run of characters within **s**.

3. What Do We Need to Find?

The **maximum number** of such substrings you can select at once (not the maximum total length — just the count).

Function Signature

■ C++

```
class Solution {
public:
    int maxPalindromes(string s, int k) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input: s = "abaccdbbd", k = 3  
Output: 2
```

We can select "aba" (positions 0-2) and "dbbd" (positions 5-8): both are palindromes, both have length ≥ 3 , and they don't overlap. It can be shown no selection of 3 or more valid substrings exists $\rightarrow$ **2**.

Example 2 — nothing qualifies

```
Input: s = "adbcda", k = 2  
Output: 0
```

There is no palindromic substring of length ≥ 2 anywhere in this string $\rightarrow$ **0**.

Constraints

- $1 \leq k \leq s.length \leq 2000$
- s consists of lowercase English letters.

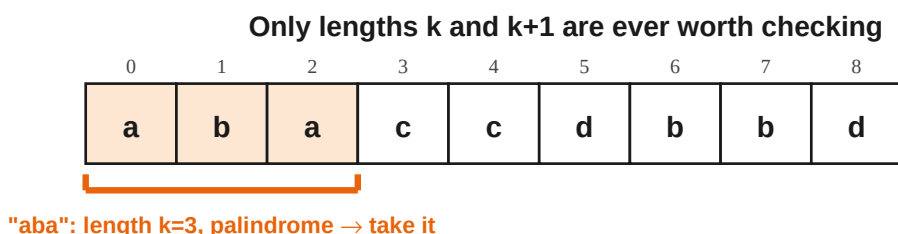
n up to 2000 rules out anything exponential, and even a naive $O(n^3)$ (checking every possible substring's palindrome status from scratch) is risky at 8×10^9 worst-case character comparisons — the intended solution needs $O(n^2)$ or better.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Set up a standard "count so far" DP.** Let $dp[i]$ = the maximum number of valid substrings selectable using only the prefix $s[0..i-1]$. The answer is $dp[n]$. At every position, you can always carry forward $dp[i] = dp[i-1]$ (simply don't use $s[i-1]$ as the end of anything).
- **Step 2 — Also try ending a valid substring right at position i.** If some substring ending at index $i-1$, with length $\geq k$, is a palindrome, and it starts at index $start$, then $dp[i]$ can also be $dp[start] + 1$ (use that substring, then whatever was optimal before it started).
- **Step 3 — Realize you don't need to check every possible length.** Suppose the shortest valid (length $\geq k$) palindrome ending at position i has length $L > k+1$. It turns out you can always find a *shorter* valid palindrome ending at some position $\leq i$ that achieves at least as good a count — using the longer one never helps and can only waste room. This is provable by induction on the structure of palindromes (stripping matching outer characters from a length- L palindrome, $L \geq k+2$, still leaves a valid, shorter palindrome candidate reachable by the same DP).
- **Step 4 — So only two lengths ever matter at each position: k and $k+1$.** For every ending position i , just check whether $s[i-k..i-1]$ (length k) is a palindrome, and separately whether $s[i-k-1..i-1]$ (length $k+1$) is a palindrome. If either is, it gives a candidate transition — and checking both, rather than only one, is needed because k and $k+1$ have different parities, so together they can catch every shortest-possible palindrome regardless of whether its center is on a character or between two characters.
- **Step 5 — Make palindrome checks $O(1)$.** Precompute a table $isPal[i][j]$ = whether $s[i..j]$ is a palindrome, filled in by increasing substring length using $isPal[i][j] = (s[i]==s[j])$ and $isPal[i+1][j-1]$. With that table ready, every check in Step 4 becomes an $O(1)$ lookup instead of an $O(n)$ character-by-character comparison.

Why Only Two Lengths Matter



A longer valid palindrome ending at the same spot never beats using the shortest one — it only eats more room for later picks.

Figure 1 — for $s = "abacccbbd"$, $k = 3$, the shortest valid palindrome ending at index 2 is "aba" itself (length exactly k); no longer candidate ending there could ever do better.

DP Transition Flow (per position i)

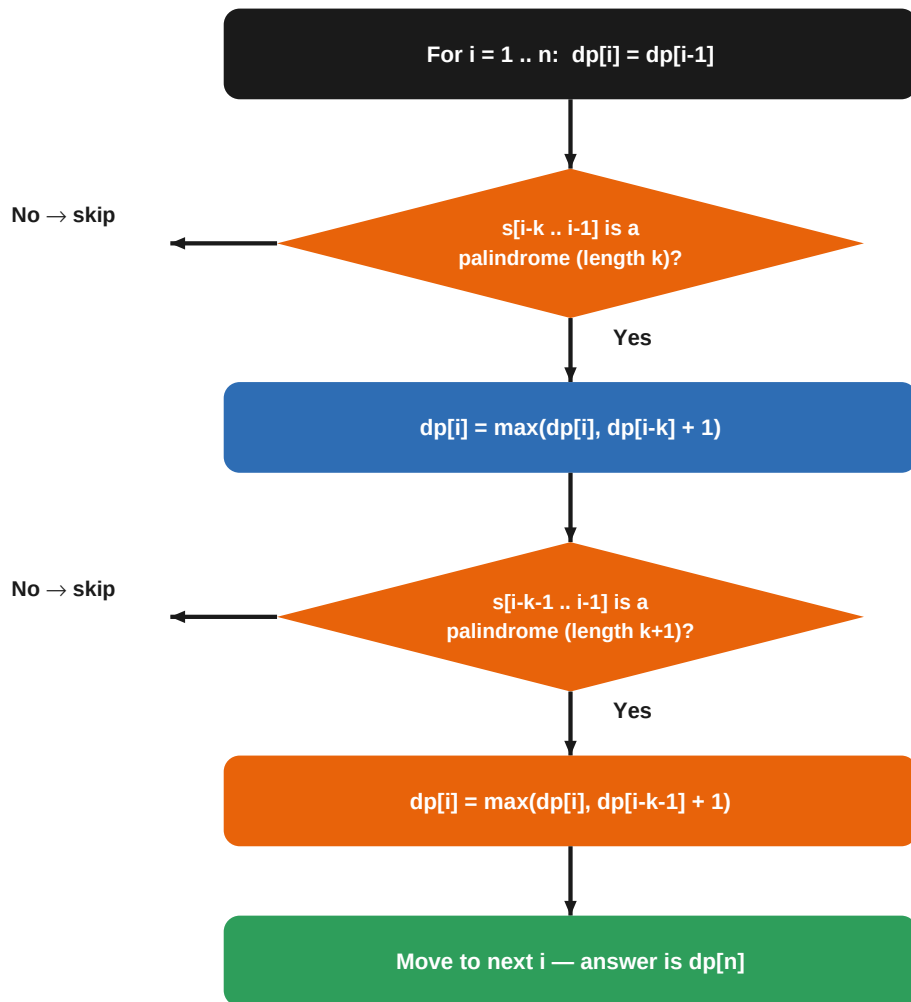


Figure 2 — at every position i , try carrying $dp[i-1]$ forward, then try both the length- k and length- $k+1$ palindrome transitions.

Which Algorithm Is Best?

Precompute the $O(n^2)$ palindrome table once, then run the $O(n)$ DP over positions, checking only two candidate lengths (k and $k+1$) at each one, each an $O(1)$ table lookup. Total: $O(n^2)$ time and space, dominated entirely by building the palindrome table — about 4 million operations at $n = 2000$, comfortably fast. The brute-force and "better" tiers below show what happens without the length- $k/k+1$ insight: either full exponential search, or an $O(n^2)$ DP that still has to check every possible length at every position, costing an extra factor of n .

Approach 1 — Brute Force (Try Every Possible Substring at Every Position)

Idea: At every position, either skip it entirely, or try starting a valid substring there of every possible length from k up to whatever remains, checking each candidate for the palindrome property directly, and recursing on whatever's left afterward.

Steps

- Recurse with (position i , count so far).
- Record the best count seen at every recursive call.
- Try every length from k to $n-i$ as a candidate substring starting at i ; if it's a palindrome, recurse from the position right after it with $\text{count}+1$.
- Also try simply moving to $i+1$ without selecting anything starting at i .
- Return the best count found across the whole search.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isPalindrome(string& s, int i, int j) {
        while (i < j) {
            if (s[i] != s[j]) return false;
            i++; j--;
        }
        return true;
    }

    int best = 0;

    // Try every possible next substring (any start, any length >= k) at each position.
    void solve(string& s, int k, int i, int count) {
        best = max(best, count);
        int n = s.size();
        for (int len = k; i + len <= n; len++) {
            if (isPalindrome(s, i, i + len - 1)) {
                solve(s, k, i + len, count + 1);
            }
        }
        if (i < n) solve(s, k, i + 1, count); // also try just skipping this position
    }

    int maxPalindromes(string s, int k) {
        best = 0;
        solve(s, k, 0, 0);
        return best;
    }
};

int main() {
    string s = "abaccdbbd";
    int k = 3;
    Solution sol;
    cout << sol.maxPalindromes(s, k) << endl; // Expected output: 2
    return 0;
}
```

■ Python


```

class Solution:
    def isPalindrome(self, s, i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i += 1
            j -= 1
        return True

    def maxPalindromes(self, s, k):
        n = len(s)
        self.best = 0

        # Try every possible next substring (any start, any length >= k) at each position.
        def solve(i, count):
            self.best = max(self.best, count)
            for length in range(k, n - i + 1):
                if self.isPalindrome(s, i, i + length - 1):
                    solve(i + length, count + 1)
            if i < n:
                solve(i + 1, count)          # also try just skipping this position

        solve(0, 0)
        return self.best

if __name__ == "__main__":
    s, k = "abaccdbbd", 3
    print(Solution().maxPalindromes(s, k))    # Expected output: 2

```

■ JavaScript

```

class Solution {
    isPalindrome(s, i, j) {
        while (i < j) {
            if (s[i] !== s[j]) return false;
            i++; j--;
        }
        return true;
    }

    maxPalindromes(s, k) {
        const n = s.length;
        let best = 0;

        // Try every possible next substring (any start, any length >= k) at each position.
        const solve = (i, count) => {
            best = Math.max(best, count);
            for (let len = k; i + len <= n; len++) {
                if (this.isPalindrome(s, i, i + len - 1)) {
                    solve(i + len, count + 1);
                }
            }
            if (i < n) solve(i + 1, count);    // also try just skipping this position
        };

        solve(0, 0);
        return best;
    }
}

const s = "abaccdbbd", k = 3;
console.log(new Solution().maxPalindromes(s, k));    // Expected output: 2

```

Complexity: Exponential: at each position there are up to $O(n)$ length choices plus a skip option, compounding across up to n positions. With n up to 2000, this is hopelessly slow — usable only for illustrating the problem on tiny strings.

Approach 2 — Better (Full DP, but Still Checking Every Length at Every Position)

Idea: Set up the DP from Steps 1–2 of the workflow correctly, and make palindrome checks $O(1)$ with a precomputed table (Step 5) — but without yet applying the length- $k/k+1$ shortcut from Step 3–4, so every position still checks every possible starting point for a valid substring ending there.

Steps

- Build the $O(n^2)$ palindrome lookup table exactly as in Step 5.
- For each position i from 1 to n : start with $dp[i] = dp[i-1]$ (skip option).
- Try every possible start position from 0 to $i-k$; if $s[start..i-1]$ is a palindrome (an $O(1)$ table lookup), update $dp[i]$ with $dp[start] + 1$ if better.
- Return $dp[n]$ once every position has been processed.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxPalindromes(string s, int k) {
        int n = s.size();

        // Precompute isPal[i][j]: whether s[i..j] is a palindrome —  $O(1)$  lookups later.
        vector<vector<bool>> isPal(n, vector<bool>(n, false));
        for (int i = 0; i < n; i++) isPal[i][i] = true;
        for (int i = 0; i + 1 < n; i++) isPal[i][i+1] = (s[i] == s[i+1]);
        for (int len = 3; len <= n; len++) {
            for (int i = 0; i + len - 1 < n; i++) {
                int j = i + len - 1;
                isPal[i][j] = (s[i] == s[j]) && isPal[i+1][j-1];
            }
        }

        vector<int> dp(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            dp[i] = dp[i - 1];
            // Check EVERY possible length  $\geq k$  ending at position  $i$  (not just  $k$  and  $k+1$ ).
            for (int start = 0; start <= i - k; start++) {
                if (isPal[start][i - 1]) {
                    dp[i] = max(dp[i], dp[start] + 1);
                }
            }
        }
        return dp[n];
    }
};

int main() {
    string s = "abaccdbbd";
    int k = 3;
    Solution sol;
    cout << sol.maxPalindromes(s, k) << endl;    // Expected output: 2
    return 0;
}
```

■ Python

```
class Solution:
    def maxPalindromes(self, s, k):
```



```

n = len(s)

# Precompute isPal[i][j]: whether s[i:j+1] is a palindrome — O(1) lookups later.
isPal = [[False] * n for _ in range(n)]
for i in range(n):
    isPal[i][i] = True
for i in range(n - 1):
    isPal[i][i + 1] = (s[i] == s[i + 1])
for length in range(3, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        isPal[i][j] = (s[i] == s[j]) and isPal[i + 1][j - 1]

dp = [0] * (n + 1)
for i in range(1, n + 1):
    dp[i] = dp[i - 1]
    # Check EVERY possible length >= k ending at position i (not just k and k+1).
    for start in range(0, i - k + 1):
        if isPal[start][i - 1]:
            dp[i] = max(dp[i], dp[start] + 1)

return dp[n]

if __name__ == "__main__":
    s, k = "abaccdbbd", 3
    print(Solution().maxPalindromes(s, k))    # Expected output: 2

```

■ JavaScript

```

class Solution {
    maxPalindromes(s, k) {
        const n = s.length;

        // Precompute isPal[i][j]: whether s[i..j] is a palindrome — O(1) lookups later.
        const isPal = Array.from({ length: n }, () => new Array(n).fill(false));
        for (let i = 0; i < n; i++) isPal[i][i] = true;
        for (let i = 0; i + 1 < n; i++) isPal[i][i + 1] = s[i] === s[i + 1];
        for (let len = 3; len <= n; len++) {
            for (let i = 0; i + len - 1 < n; i++) {
                const j = i + len - 1;
                isPal[i][j] = s[i] === s[j] && isPal[i + 1][j - 1];
            }
        }

        const dp = new Array(n + 1).fill(0);
        for (let i = 1; i <= n; i++) {
            dp[i] = dp[i - 1];
            // Check EVERY possible length >= k ending at position i (not just k and k+1).
            for (let start = 0; start <= i - k; start++) {
                if (isPal[start][i - 1]) {
                    dp[i] = Math.max(dp[i], dp[start] + 1);
                }
            }
        }
        return dp[n];
    }
}

const s = "abaccdbbd", k = 3;
console.log(new Solution().maxPalindromes(s, k)); // Expected output: 2

```

Complexity: Building the palindrome table costs $O(n^2)$. The DP itself now costs $O(n^2)$ as well, since each of the n positions scans up to n possible start points — but combined with the table, that's still $O(n^2)$ overall, and in practice a real constant-factor improvement over Approach 1. The remaining inefficiency is doing an $O(n)$ scan per position when, as Step 3 shows, only 2 of those n candidate lengths can ever actually matter.

Approach 3 — Optimal (Only Check Lengths k and k+1 at Each Position)

Idea: Apply the Step 3–4 lemma directly: at every position, the only two candidate substring lengths that could ever be part of an optimal selection are the shortest legal length (k) and the next one up (k+1) — checking both covers every case regardless of parity. This collapses the per-position work from an $O(n)$ scan down to two $O(1)$ table lookups.

Steps

- Build the same $O(n^2)$ palindrome lookup table as Approach 2.
- For each position i from 1 to n : start with $dp[i] = dp[i-1]$ (skip option), exactly as before.
- Check only $s[i-k..i-1]$ (length exactly k); if it's a palindrome, try $dp[i-k] + 1$.
- Check only $s[i-k-1..i-1]$ (length exactly $k+1$); if it's a palindrome, try $dp[i-k-1] + 1$.
- Return $dp[n]$ — no other lengths ever need to be examined.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxPalindromes(string s, int k) {
        int n = s.size();

        // Precompute isPal[i][j]: whether s[i..j] is a palindrome.
        vector<vector<bool>> isPal(n, vector<bool>(n, false));
        for (int i = 0; i < n; i++) isPal[i][i] = true;
        for (int i = 0; i + 1 < n; i++) isPal[i][i+1] = (s[i] == s[i+1]);
        for (int len = 3; len <= n; len++) {
            for (int i = 0; i + len - 1 < n; i++) {
                int j = i + len - 1;
                isPal[i][j] = (s[i] == s[j]) && isPal[i+1][j-1];
            }
        }

        vector<int> dp(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            dp[i] = dp[i - 1];

            if (i >= k && isPal[i - k][i - 1]) { // check length k
                dp[i] = max(dp[i], dp[i - k] + 1);
            }
            if (i >= k + 1 && isPal[i - k - 1][i - 1]) { // check length k+1
                dp[i] = max(dp[i], dp[i - k - 1] + 1);
            }
        }
        return dp[n];
    }
};

int main() {
    string s = "abaccdbbd";
    int k = 3;
    Solution sol;
    cout << sol.maxPalindromes(s, k) << endl; // Expected output: 2
    return 0;
}
```

■ Python

```

class Solution:
    def maxPalindromes(self, s, k):
        n = len(s)

        # Precompute isPal[i][j]: whether s[i:j+1] is a palindrome.
        isPal = [[False] * n for _ in range(n)]
        for i in range(n):
            isPal[i][i] = True
        for i in range(n - 1):
            isPal[i][i + 1] = (s[i] == s[i + 1])
        for length in range(3, n + 1):
            for i in range(n - length + 1):
                j = i + length - 1
                isPal[i][j] = (s[i] == s[j]) and isPal[i + 1][j - 1]

        dp = [0] * (n + 1)
        for i in range(1, n + 1):
            dp[i] = dp[i - 1]

            if i >= k and isPal[i - k][i - 1]:                # check length k
                dp[i] = max(dp[i], dp[i - k] + 1)
            if i >= k + 1 and isPal[i - k - 1][i - 1]:        # check length k+1
                dp[i] = max(dp[i], dp[i - k - 1] + 1)

        return dp[n]

if __name__ == "__main__":
    s, k = "abaccdbbd", 3
    print(Solution().maxPalindromes(s, k))    # Expected output: 2

```

JavaScript

```

class Solution {
    maxPalindromes(s, k) {
        const n = s.length;

        // Precompute isPal[i][j]: whether s[i..j] is a palindrome.
        const isPal = Array.from({ length: n }, () => new Array(n).fill(false));
        for (let i = 0; i < n; i++) isPal[i][i] = true;
        for (let i = 0; i + 1 < n; i++) isPal[i][i + 1] = s[i] === s[i + 1];
        for (let len = 3; len <= n; len++) {
            for (let i = 0; i + len - 1 < n; i++) {
                const j = i + len - 1;
                isPal[i][j] = s[i] === s[j] && isPal[i + 1][j - 1];
            }
        }

        const dp = new Array(n + 1).fill(0);
        for (let i = 1; i <= n; i++) {
            dp[i] = dp[i - 1];

            if (i >= k && isPal[i - k][i - 1]) {                // check length k
                dp[i] = Math.max(dp[i], dp[i - k] + 1);
            }
            if (i >= k + 1 && isPal[i - k - 1][i - 1]) {        // check length k+1
                dp[i] = Math.max(dp[i], dp[i - k - 1] + 1);
            }
        }
        return dp[n];
    }
}

const s = "abaccdbbd", k = 3;
console.log(new Solution().maxPalindromes(s, k));    // Expected output: 2

```

Complexity: Building the palindrome table still costs $O(n^2)$, but the DP loop itself is now $O(n)$ with $O(1)$ work per position — $O(n^2)$ overall, dominated entirely by table construction rather than by the DP transitions. At $n = 2000$ that's about 4 million operations, comfortably fast, and the per-position logic is now a fixed, tiny amount of work instead of scanning up to n candidates.

Side-by-Side Comparison

1. Brute Force	Recursively try every start/length combination at every position	Exponential — never finishes	Stack overflow
2. Better	Full DP with a palindrome table, but every length checked	$O(n^2)$	$O(n^2)$ for the palindrome table
3. Optimal	Same DP and table, but only lengths k and $k+1$ checked	$O(n^2)$ dominated by table build	$O(n^2)$ for the palindrome table

Key Takeaways to Remember

- "**Maximize the count of non-overlapping valid pieces**" is a signal for a $dp[i]$ = best-count-using-prefix- i DP — always carry forward $dp[i-1]$, then try "end something valid right here" as an alternative.
- When maximizing a *count* (not total length or value), the **shortest** valid option at each point is often provably at least as good as any longer one — using less "room" never hurts and can only help later picks. Always ask whether a problem secretly only needs the minimal valid choice at each step.
- Checking two adjacent lengths (here, k and $k+1$) rather than one is a common pattern when a structural property (here, palindromes) behaves differently depending on parity — one length alone can't always find the shortest match.
- Precomputing an $O(n^2)$ palindrome table via increasing substring length ($isPal[i][j] = (s[i]==s[j])$ and $isPal[i+1][j-1]$) is the standard way to turn repeated palindrome checks into $O(1)$ lookups.
- All three tiers were validated against **Example 1** ($s = \text{"abaccd bbd"}$, $k = 3 \rightarrow 2$) across C++, Python, and JavaScript, and checked against Example 2 ($s = \text{"adbcda"}$, $k = 2 \rightarrow 0$) as well, plus 300 randomized cross-checks against the brute-force solution.